System Design – Interview Notes

Part 1 (1–44): your notes · Part 2 (45–97): advanced system design · Part 3 (98–104): low-level design · Part 4 (105–106): interview prep. Highlighted = interview keyword. *Green italics = example.* ✔ corrected = fixed from the original notes.

Part 1 – Basics: from one server to a scalable app

Suppose you have an application (Amazon) and you want to scale it up.

1. You buy an AWS server with a public IP address so that people can reach your application.

2. It is a cloud server running 24×7 with a public IP address.

3. Your client (the user) sends a request to that IP address to use your app.

4. An IP address is a number that identifies your application on the huge internet network.

5. An IP address looks like 10.2.3.4, which is hard to remember and not human-friendly. So we have DNS, which works like a phonebook: you give it a name and it finds the matching IP address. `www.amazon.com → 10.2.3.4`

6. When you ask the DNS server for the IP of a name like amazon.com, it returns the IP address. This process is called DNS resolution.

Scaling

7. Traffic grows and your server crashes, so you upgrade it from 2 CPU / 4 GB RAM to 64 CPU / 128 GB RAM, and it copes. Later the user base grows further and you need something like 6000 CPU / 128K GB RAM. That works, but it's a huge cost, and you only need that much during a sale or New Year. The rest of the time those resources sit unused. Making the same machine bigger and bigger to handle requests is called Vertical Scaling.

8. In vertical scaling, you have to shut the server down to add hardware and restart it afterwards, which can cause business loss. This is called Downtime.

9. Horizontal Scaling helps here: you add a second machine with the extra resources, and it starts with no downtime on the existing server.

10. Problem: DNS only has the address of your existing server. New servers won't be known to DNS, so DNS sends every request to the old server.

11. These extra servers are replicas. For horizontal scaling you place a Load Balancer between DNS and your servers.

12. DNS stores the IP of the load balancer instead of the IP of your application server.

13. The load balancer spreads user requests across the servers. It uses an algorithm such as Round Robin (each server in turn) to share the load equally.

14. When you add a new server, you register it with the load balancer. The LB runs health checks on the new server for a while, and once it **looks healthy** the LB starts sending it requests. If a server becomes unhealthy, the LB stops sending it traffic.

15. The load balancer itself could crash, so it is given plenty of resources and runs on multiple machines and threads.

Microservices & API Gateway

16. Your Amazon app has several microservices (Payments, Orders and so on), each with several instances of its own. For horizontal scaling, an API Gateway sends each request to the load balancer of the right service. `/orders → Orders LB` `/payment → Payment LB`. Each LB then spreads the requests across that service's servers.

17. The servers/replicas we use for horizontal scaling are EC2 instances, which run their own VM.

18. Now the **API Gateway's** IP is registered with DNS instead of a load balancer's.

19. You can define rules and authentication at the API gateway level.

20. Once this architecture is in place, you may need batch processing (e.g. big file drops). For that you run background services such as an email service or worker.

Sync vs Async, Queues & Pub-Sub

21. Suppose that after a payment for an order, you want to send the order details to the user by email. The payment service waits for the email service to finish before responding. This is Synchronous communication. It's not ideal for big applications because the payment is stuck waiting.

22. To avoid this we use Asynchronous communication, most commonly through a Queue system.

23. The payment service pushes an event into the queue and the email service consumes the events one by one. The email service can be scaled horizontally when needed, which increases Parallelism. External services like Gmail apply a rate limit per second on their side, so running several email services helps with parallelism and horizontal scaling.

24. In AWS this is SQS (Simple Queue Service).

25. A queue can deliver events in two ways:

- Pull mechanism: the email service fetches events from the queue by **polling** it, e.g. every 5–10 seconds, to see if anything is new.
- Push mechanism: the queue itself calls the email service for each event.
- SQS uses **pull**, with either short polling (returns immediately, even if empty) or long polling (waits **up to 20 seconds** for a message to arrive, so there are fewer empty calls). ✔ corrected

26. A new requirement: when a payment happens you want several things to happen alongside the email. One event triggers several actions, so we use the Pub-Sub model, here with **SNS** (Simple Notification Service). The payment service **publishes** an event to SNS, and the email, WhatsApp and SMS services are **subscribed** to it. This is called event-driven architecture.

27. In SQS, a message read by one service is gone, so no other service can get it. SNS solves this by delivering the same event to every subscriber.

28. In a queue system there is acknowledgement: if a consumer takes an event but fails to process it, the event is retried and eventually moved to a DLQ (Dead Letter Queue) to be handled later. SNS is different: it **doesn't store messages**. It retries delivery a few times (and can send failures to a DLQ), but subscribers can't come back later and read an event. ✔ corrected

29. To get both, we use the Fan-out architecture, which is very common. SNS pushes each event into several SQS queues, so each service works independently with storage and retries.

```
Payment → SNS → [WhatsApp Queue] [Email Queue] [SMS Queue] (SQS)
```

30. A big application can face a DDoS attack, where a user sends a flood of requests in a short time. This is where Rate Limiting comes in. Two common algorithms:

- Token Bucket: a bucket refills with tokens at a fixed rate, and each request uses one token. No tokens left → the request is rejected. Allows short bursts.
- Leaky Bucket: requests enter a bucket and leave it at a fixed, steady rate. When the bucket is full, new requests are dropped. Smooths traffic.

Database Scaling, Caching & CDN

31. A big application gets an overwhelming number of DB requests. To keep the database fast, we use read replicas: copies of the DB that handle read requests that don't need real-time data. Writes go to the Master Node (the main DB), and replicas may lag slightly behind (a delay in data sync).

32. To avoid DB calls altogether we use Caching, which serves frequently accessed data from memory instead of calling the DB.

33. Your load balancer, services and DB are in the US, and a user in India checks a product. This adds big latency. To speed up responses and reduce DB and service calls, we use a CDN (Content Delivery Network): CDN servers placed around the world. With Anycast, all of them share the same IP address, and a user's request reaches the nearest CDN server. If that server already has the data cached, it responds; if not, it forwards the request to the load balancer and service, then caches the result so the next request is answered quickly.

Serverless vs Stateful

34. As the app grows there are two options: a stateful architecture, where you manage load balancing and resources yourself, or a stateless (serverless) one, where you only manage the code and database and an external service like **AWS Lambda** handles auto-scaling. Each request triggers a Lambda function as a separate task.

- **Pros**: cheap (1M requests free, then about $0.20 per 1M requests).
- **Cons**:
  → Cold start: the first request takes longer.
  → **Duration limit**: a Lambda can run for at most **15 minutes**. If it's called through API Gateway, the HTTP response must come back within ~29 seconds. ✔ corrected
  → **DDoS**: a flood of requests means a flood of Lambda invocations, and the bill goes up with it.
  → Vendor lock-in: you gradually start using other services from the same vendor (SQS, S3, API Gateway, routing), so leaving later becomes costly and hard.

35. The architecture we were building before serverless is the stateful architecture.

36. We prefer stateful when we want full control over scaling the application and want to avoid vendor lock-in.

Containers, Orchestration & Kubernetes

37. When a service needs to scale, you have to replicate its whole environment (OS, dependencies, packages, codebase). Different machines bring issues like package mismatches or OS differences, the classic "It works on my machine" problem.

- Virtualization: create a full virtual machine (OS + packages + codebase) on the server. Heavy, and slow to start.
- Containerization: a lightweight package of just your code + packages + dependencies that runs the same anywhere. A container **shares the host machine's OS kernel** instead of carrying its own full OS, which is why it's lightweight and starts in seconds. ✔ corrected *Docker.*
- You can start or delete several containers on one server as needed, which helps when load goes up or down. Several containers running across several servers form a Cluster.

38. With many containers across many servers, you need something to manage them. That's the Orchestration concept.

39. When you change code, you push the new build, and the orchestration tool starts new containers with the updated build and removes the old ones gradually. This deployment strategy is called Rolling Updates.

40. Google created a tool named Borg to manage its huge cluster infrastructure. It worked as a container orchestrator, the "brain" handling auto-scaling, auto load balancing and so on. Google then built and donated Kubernetes, inspired by Borg, which is now the most widely used container orchestration tool.

41. Kubernetes definition: Kubernetes manages the containers inside a cluster of servers: how to update and replace them so that you have **zero downtime**, how to scale them and how to restart failed ones.

Proxies & Deployment

42. Forward Proxy:

- Works on behalf of the **client**. It sits between the client and the server, and hides the client's identity from the destination server.
- *A company proxy controls which websites employees can visit.* `Employee → Forward Proxy → Website`

43. Reverse Proxy:

- Works on behalf of the **server**. It spreads requests, improves security, handles load balancing and hides the backend servers' details.
- *An e-commerce site gets thousands of requests. Nginx (the reverse proxy) receives them and spreads them across several backend servers so no single server is overloaded.* `Users → Reverse Proxy → Backend Servers`

44. Blue-Green Deployment is a deployment strategy where we keep two identical environments:

- **Blue** runs the current production version; **Green** runs the new version.
- We deploy and test the new version in Green, then switch production traffic to it. If any issue appears, we roll back quickly by sending traffic back to Blue. This minimizes downtime and deployment risk.

Part 2 – Advanced Concepts

A. Databases

45. Your Amazon app has to store users, orders and products somewhere. The first choice is SQL vs NoSQL.

- **SQL** (MySQL, Postgres): data lives in tables with a fixed structure, and tables can be joined. Follows ACID: a transaction either fully happens or doesn't happen at all. *Orders, payments, bank balance.*
- **NoSQL** (MongoDB, DynamoDB, Cassandra): flexible structure (JSON-like) and easy to spread over many servers. Follows BASE: data may be slightly out of date for a moment but becomes correct shortly after. *Product catalog, cart, chat messages, logs.*

46. CAP Theorem: if the network between your DB servers breaks (**P**artition), you can keep only one of these two:

- **Consistency** → refuse requests rather than return old data. *Banking: wrong data is worse than an error.*
- **Availability** → always respond, even with slightly old data. *Instagram likes: showing 99 instead of 100 is fine.*

47. Strong consistency: after a write, every read immediately sees the new value. Eventual consistency: replicas may show the old value for a few milliseconds or seconds, then catch up. *The read-replica delay from point 31 is eventual consistency.*

48. Indexing: without an index, finding a user by email means checking every row (a full table scan). An index is like the index at the back of a book; it points straight to the right row.

- Trade-off: reads get much faster, but every insert or update must also update the index, so writes get slower. Index only the columns you search on.

49. Replication types (point 31 was master/replica):

- Leader–Follower: only the leader accepts writes and copies them to followers. Followers serve reads. Simple and the most common.
- Multi-Leader: each region has its own leader accepting writes. Faster for global users, but two regions can edit the same data, so you need conflict resolution.
- Leaderless (Cassandra, DynamoDB): writes go to several nodes at once. The Quorum rule `W + R > N` means that if enough nodes confirm the write (W) and you read from enough nodes (R), at least one of them has the latest data. *N=3, W=2, R=2.*

50. Your orders table grows to billions of rows, and one DB server can no longer store it or handle all the reads and writes. Four related ideas solve this. They are often confused, so learn them together.

(a) Partitioning: split one big table into smaller pieces called **partitions**. The pieces *can stay on the same server*.

- Horizontal partitioning: split by **rows**; every partition has the same columns. *orders_2024, orders_2025. A query for this month's orders searches one small partition instead of the whole table.*
- Vertical partitioning: split by **columns**. *Basic user info (name, email) in one table; large rarely-used data (profile photo, bio) in another.*

(b) Sharding: horizontal partitioning where **each piece lives on a different server**. Each server is a **shard**: same table structure, different rows.

- A shard key decides which shard a row goes to, usually userId. *Users 1–1M → Shard 1, 1M–2M → Shard 2.*

- Strategies: **Range-based** (by ID or date ranges; simple but can be uneven), **Hash-based** (`hash(userId) % N` ; spreads data evenly), **Geo-based** (Indian users on the India shard; low latency).
- Benefit: both **storage and write load** are spread over many machines.
- Problems: hot shard (a celebrity's shard gets overloaded); **cross-shard queries** (a join across shards is slow, so pick a shard key that keeps related data together); **re-sharding** when adding servers (solved by consistent hashing, point 51).

(c) Replication: keep **copies of the same data** on several servers. It doesn't add storage capacity; it adds **safety and read capacity**.

- If one server dies, a copy takes over (**high availability**). Reads are spread across the copies (**read scaling**). Types are in point 49.

(d) Clustering: a **group of servers working together** that the app sees as one database. The cluster handles where data lives, copying it and failover.

- A cluster usually uses **sharding and replication together**. *A MongoDB cluster with 3 shards, each shard having 1 leader + 2 replicas = 9 servers. The app just connects to "the DB".*
- Examples: MongoDB sharded cluster, Cassandra cluster, Redis Cluster. (Same idea as the server cluster in point 37, applied to databases.)

```
App → Cluster → [Shard 1: Leader + 2 Replicas] [Shard 2: Leader + 2 Replicas] [Shard 3: Leader + 2 Replicas]
```

Concept	What it does	Data on each server	Solves
Partitioning	Splits a table into pieces	— (may be one server)	Faster queries on big tables
Sharding	Puts those pieces on different servers	**Different** data	Storage + write scaling
Replication	Copies data to several servers	**Same** data	Availability + read scaling
Clustering	Groups the servers into one system	Both	Managing it all as one DB

Interview line: *"Shard to scale writes and storage, replicate each shard for availability and reads, and run it all as a cluster."*

51. Consistent Hashing: with `hash % N` , going from 4 to 5 DBs changes N, so almost every key moves to a different server, which is a huge data migration.

- Fix: place servers on a circle (a **hash ring**). Each key is hashed onto the circle and stored on the next server clockwise.
- Adding a server only takes keys from its neighbour, so only a small share of the data moves. Virtual nodes put each server at several points on the ring so the load stays even.

B. Caching (deep dive on point 32)

52. How data gets into the cache:

- Cache-Aside (most common): the app checks the cache first. On a miss, it reads the DB and saves the result in the cache for next time.

- Write-Through: every write goes to the cache and the DB together. The cache is always fresh, but writes are slower.
- Write-Back: write only to the cache and save to the DB later in batches. Very fast, but if the cache crashes before saving, that data is lost.

53. When the cache is full, remove something (eviction): LRU removes what hasn't been used for the longest time (most common). LFU removes what is used least often. TTL means every key expires after a set time.

54. Cache problems:

- Stale data: the DB is updated but the cache still holds the old value. Fix: delete the cache key when the data changes, and set a TTL.
- Cache stampede: a popular key (say, the iPhone product page) expires and 10,000 requests miss the cache and hit the DB at the same time. Fix: let only one request rebuild the cache while the others wait.
- Tool: Redis, an in-memory store that is very fast and supports lists, sets and counters.

C. Load Balancer & Rate Limiting (add to points 13 & 30)

55. More LB algorithms:

- Least Connections: send the request to the server with the fewest active requests. Good when some requests take much longer than others.
- Weighted Round Robin: a server with twice the power gets twice the requests.
- IP Hash: the same user always reaches the same server. Useful when the server keeps that user's session.

56. L4 vs L7 Load Balancer: L4 only looks at IP and port, like a postman who reads the address but not the letter, so it's very fast. L7 reads the HTTP request itself (URL, headers), so it can route smartly. `/orders` → *Order service,* `/payments` → *Payment service.*

57. More rate-limiting algorithms:

- Fixed Window: allow 100 requests per minute and reset the count at :00. Problem: a user can send 100 at 0:59 and 100 more at 1:00.
- Sliding Window: always count the requests in the *last 60 seconds* from now, which removes that burst. Most common in practice.
- Counts are stored in Redis. When the limit is hit, return **HTTP 429 (Too Many Requests)**.

D. Communication & Networking

58. TCP vs UDP: TCP makes sure every packet arrives, in order, and resends lost ones; it's reliable but slower. UDP just sends and doesn't check; it's fast but packets can be lost. *TCP: websites, payments. UDP: video calls, online games, where a lost frame doesn't matter.*

59. HTTP/1.1 vs 2 vs 3: 1.1 handles one request at a time per connection. HTTP/2 sends many requests together over one connection, so pages load faster. HTTP/3 runs on UDP (QUIC), so it's faster on weak mobile networks. HTTPS = HTTP + encryption (TLS).

60. API styles:

- REST: URLs plus HTTP methods (`GET /orders/5`), returns JSON. Simple; used for public APIs.
- GraphQL: the client asks for exactly the fields it needs in one request. Avoids fetching too much data. *A mobile app needs only the name and price.*
- gRPC: sends compact binary data instead of JSON, so it's very fast. Used **between internal microservices**.

61. Real-time updates (the server needs to send data without being asked):

- Long Polling: the client asks and the server holds the request open until new data arrives, then the client asks again.
- SSE (Server-Sent Events): a one-way stream from server to client. *Live cricket score, notifications.*
- WebSockets: a connection that stays open and lets both sides send at any time. *WhatsApp chat, multiplayer games.*

E. Messaging (deep dive on points 22–29)

62. Kafka: like a queue, but messages are **not deleted after reading**. They are kept on disk for days, so several services can read the same events, and you can re-read (**replay**) old events.

- A topic is split into partitions so that several consumers can read in parallel. Order is guaranteed only inside one partition.
- **SQS** = a to-do list (a task is done, then removed). **Kafka** = a diary of events (a permanent record many services can read). *Kafka: order events read by the analytics, recommendation and email services.*

63. Delivery guarantees: At-most-once means a message may be lost but is never repeated. At-least-once means it's never lost but may arrive twice (the most common). Exactly-once is ideal but hard and expensive.

64. Because messages can arrive twice, use Idempotency: doing the same operation twice has the same effect as doing it once. *Each payment request carries a unique **idempotency key**. If the server sees the same key again, it returns the earlier result instead of charging again.*

Networking Recap – OSI Model (builds on section D)

65. OSI Model: 7 layers that describe how data travels from an app on one machine to an app on another. Each layer does one job. On the sending side, each layer wraps the data and passes it down. On the receiving side, the layers unwrap it in reverse.

#	Layer	What it does	Examples
7	Application	What the app uses to talk	HTTP, DNS, SMTP
6	Presentation	Format, encryption, compression	TLS/SSL, JSON, JPEG
5	Session	Opens, keeps and closes a connection	Sessions, sockets
4	Transport	Reliable delivery between ports	TCP, UDP, ports
3	Network	Routing between networks using IP	IP, routers
2	Data Link	Delivery within the local network using MAC addresses	Ethernet, switches
1	Physical	Raw bits over a wire or the air	Cables, Wi-Fi signals

- Mnemonic (7→1): **"All People Seem To Need Data Processing"**.

- *Opening amazon.com: an HTTP request (7) is encrypted by TLS (6), sent over TCP port 443 (4), routed using IP (3), framed with MAC addresses (2), and sent as Wi-Fi signals (1).*
- This is where the names **L4 vs L7 load balancer** (point 56) come from, and TCP/UDP (point 58) sit at layer 4. The real internet follows the simpler 4-layer **TCP/IP model**, but interviewers ask about OSI.

F. Microservices & Distributed Patterns

66. Monolith vs Microservices: a monolith is one big app; it's easy to build and deploy, but you have to scale and redeploy the whole thing. Microservices are many small apps that scale and deploy independently, but they add network calls, harder debugging and data-sync problems. *Start with a monolith and split it when the team and traffic grow.*

67. Strangler Fig Pattern: how to move from a monolith to microservices **gradually**. Put a proxy or API Gateway in front, move one feature at a time into a new service, and route that feature's traffic to the new service. The monolith shrinks until it can be removed. *Named after a vine that slowly grows around a tree and replaces it.*

68. Database per Service: each microservice owns its own DB, and other services can reach that data only through its API, never by reading its tables directly. This keeps services independent. The cost is that a single transaction can't span services anymore, which is why Saga exists.

69. Placing an order touches the Order, Payment and Inventory services, each with its own DB, so one DB transaction can't cover all three. The first option is 2PC (Two-Phase Commit): a coordinator asks every service "Can you commit?" (prepare); if all say yes, it tells them "Commit". It is strongly consistent, but slow and blocking, and everyone is stuck if the coordinator fails. So microservices usually use **Saga** instead ↓

70. Saga Pattern: a big transaction across services is split into **a sequence of local transactions**. Each step has a compensating action that undoes it if a later step fails.

```
Create Order → Charge Payment → Reserve Inventory → Ship
```

If Reserve Inventory fails: `Refund Payment → Cancel Order` (compensate in reverse order).

- Choreography: no central controller. Each service listens for events and reacts. *OrderCreated → Payment service charges → publishes PaymentDone → Inventory service reserves…* Simple for 2–4 steps, but hard to follow the flow as it grows.
- Orchestration: a central **Saga Orchestrator** tells each service what to do next and triggers the compensations on failure. Easier to track and debug; better for complex flows. The orchestrator itself must be highly available.
- Saga gives **eventual consistency**, not ACID: the system is briefly in an in-between state (order "PENDING"). Every step must be **idempotent** (point 64), because retries happen.

71. Outbox Pattern: the problem is that a service saves an order to its DB and then publishes an "OrderCreated" event to Kafka. If it crashes between the two, the DB has the order but the event is lost (or the reverse).

- Fix: in the **same DB transaction**, save the order **and** a row in an **outbox table**. A separate process reads the outbox table and publishes the events, retrying until they succeed. Either both are saved or neither is. Often paired with Saga.

72. CQRS (Command Query Responsibility Segregation): use separate models (often separate DBs) for writing and for reading, because they have different needs. *Orders are written to Postgres, then copied to Elasticsearch for fast search and filtering. Each side scales independently.*

73. Event Sourcing: instead of storing only the current value, store every change as an event. *A bank account stores "+500, −200, +100" rather than just "balance = 400". Replay the events to get the balance. You get a full history and audit trail.*

74. Leader Election / Consensus: in a cluster, the nodes must agree on one leader, and if it dies, quickly agree on a new one, without two nodes both believing they're the leader. Algorithms: **Raft**, **Paxos**. Tools that do this for you: **ZooKeeper**, **etcd** (Kubernetes stores its state in etcd).

75. Distributed Lock: two servers try to sell the last iPhone at the same moment. A lock (in Redis or ZooKeeper) lets only one of them proceed. Always give the lock an expiry, so it is released even if the server holding it crashes.

76. Unique IDs: after sharding, each DB's auto-increment produces duplicates (two orders with ID 101).

- UUID: a random 128-bit ID that is practically always unique, but long and not sortable.
- Snowflake ID (Twitter): a 64-bit number built from `timestamp + machineId + counter`. Unique, and sorts by creation time.

77. Service Discovery: in Kubernetes, containers keep starting and stopping, so their IPs change. Services register in a **registry** (Consul, Kubernetes DNS), and others look up "order-service" by name instead of a fixed IP.

78. Sidecar Pattern: run a helper container next to each service container, in the same Kubernetes pod, to handle shared concerns such as logging, TLS, retries and metrics, so the service code stays focused on business logic. This is how a service mesh (Istio) works.

79. API Composition / BFF (Backend for Frontend): one service (or the gateway) calls several services and combines the results into a single response. A **BFF** is a separate backend per client type (mobile BFF, web BFF), each returning exactly what that client needs. *The mobile app gets a lighter product response than the website.*

G. Reliability & Fault Tolerance

80. SPOF (Single Point of Failure): any single component whose failure brings down everything. *Only one load balancer.* Fix with redundancy: keep two or more of everything, spread across **Availability Zones** (separate data centers).

81. Failover: **Active–Passive** = a backup server sits idle and takes over if the main one fails. **Active–Active** = all servers handle traffic, and if one fails the others absorb its share.

82. When a service you depend on is slow or down (say, the Payment service):

- Timeout: stop waiting after, say, 2 seconds instead of hanging forever.
- Retry with Exponential Backoff: retry after 1s, then 2s, then 4s, giving the service time to recover. Add jitter (a small random delay) so that 1,000 clients don't all retry at the same instant.
- Circuit Breaker: works like the electrical breaker at home. After repeated failures it "trips" (**Open**) and stops calling the service, returning an error or fallback immediately. After a while it lets a few test calls through (**Half-Open**); if they succeed it goes back to normal (**Closed**). This stops the failure spreading through the system.
- Graceful degradation: if the recommendation service is down, still show the product page, just without the recommendations.

83. Availability "nines": 99.9% ≈ 8.7 hours of downtime a year. 99.99% ≈ 52 minutes a year.

- SLI = what you measure (e.g. response time). SLO = your internal target (95% of requests under 200ms). SLA = the promise in the customer contract, with a penalty (refund) if broken.

84. Disaster Recovery: RPO = how much data you can afford to lose (backups every hour → you lose up to 1 hour). RTO = how long it takes to get back online.

H. Deployment (add to points 39 & 44)

85. Canary Deployment: release the new version to only 5% of users first and watch the error rate and latency. If everything looks fine, increase to 25%, 50%, then 100%. If not, roll back, having affected only a few users. *Named after the canaries miners used to detect gas early.*

86. Feature Flags: ship new code switched off and turn it on with a config switch, no redeploy needed. You can enable it for internal users first, or for 10% of users (A/B testing).

I. Observability & Security

87. Observability = knowing what's happening inside the system:

- Logs: text records of events ("Order 101 failed: payment timeout"). Tool: ELK stack.
- Metrics: numbers over time (CPU %, requests/sec, error rate), shown on dashboards with alerts. Tools: Prometheus + Grafana.
- Distributed Tracing: one request ID follows a request through all the services, showing which one was slow. Tool: Jaeger.

88. Authentication vs Authorization: Authentication = *who are you?* (login with a password or OTP). Authorization = *what are you allowed to do?* (an admin can delete products; a normal user can't).

89. Session vs JWT (token):

- **Session**: after login, the server stores your session in its memory or DB and gives you a session ID cookie. Every request means looking up the session, which is hard to share across many servers.
- JWT (JSON Web Token): after login, the server gives you a **signed token** containing your userId and role. On each request the server only checks the signature; it doesn't look anything up. That makes it **stateless**, so it scales easily (point 34).

90. Access Token vs Refresh Token: a JWT can't be cancelled before it expires. If it's stolen, the attacker can use it until then. So we use two tokens:

- Access Token: **short-lived** (about 15 min). Sent with every API call (`Authorization: Bearer <token>`). If stolen, it's useful only for a few minutes.
- Refresh Token: **long-lived** (7–30 days). Stored safely (an HttpOnly cookie) and sent **only** to `/refresh` to get a new access token when the old one expires. The server keeps a record of refresh tokens, so it can **revoke** one (on logout or a stolen device).
- **Flow**:

  ```
  Login → get both tokens → call APIs with access token → access expires (401) → send refresh token → get new access token → continue
  ```

  . The user stays logged in for days without typing a password again.

- Refresh token rotation: each refresh also issues a new refresh token and invalidates the old one, so a stolen refresh token stops working quickly.

91. OAuth 2.0: "Login with Google". The app never sees your Google password. Google asks for your permission and then gives the app a token with limited access (e.g. only your email). SSO: log in once and get access to many apps (Gmail, YouTube, Drive).

J. Storage & Special Data Structures

92. Storage types: Object storage (S3) holds files such as product images, videos and backups, each reachable by a URL. Block storage (EBS) is a hard disk attached to one server. **Rule:** store the file in S3 and only its URL in the DB, never the file itself.

93. Search: `WHERE name LIKE '%phone%'` scans every row, which is too slow for millions of products. Elasticsearch uses an Inverted Index, a map from each word to the products containing it (`"phone" → [1, 7, 42]`), like a book's index. Search becomes nearly instant and supports typos and ranking.

94. Bloom Filter: answers *"is this item in the set?"* using very little memory.

- **How it works**: a bit array (all 0s) plus a few hash functions. To add an item, hash it with each function and set those positions to 1. To check an item, hash it again: if any of those positions is 0, it is **definitely not present**; if all are 1, it is **probably present** (possibly a false positive).
- It never says "no" for an item that was added. It can occasionally say "yes" wrongly.
- *Signup: "is this username taken?" Check the Bloom filter first, and if it says "definitely not", skip the DB call. Also used by web crawlers (already visited this URL?) and Cassandra (skip disk reads).*

95. Geohash: how Uber or Zomato finds things *nearby*.

- The world map is divided into a grid, and each cell gets a short string code. Each extra character divides the cell into smaller cells, so `"tdr1"` is a big area and `"tdr1y7"` is a few streets.
- Nearby places share the same **prefix**, so "find drivers near me" becomes a simple query: `WHERE geohash LIKE 'tdr1y%'`, plus a check of the neighbouring cells. Quadtree is a similar idea: keep splitting the map into 4 squares until each square holds only a few places.

96. Trie: a tree where each node is a letter, so words with the same beginning share a path (c → a → r → *car*, *cart*, *carpet*). Used for **search autocomplete**: walk down the letters typed so far and show the top suggestions stored at that node.

97. Pagination: **Offset** (`LIMIT 10 OFFSET 10000`) means the DB still walks through 10,000 rows, which gets slow on deep pages. **Cursor** (`WHERE id > last_seen_id LIMIT 10`) jumps straight there, so it's fast. Infinite-scroll feeds (Instagram) use cursors.

Part 3 – Low-Level Design: Principles & Patterns

K. Clean Code Principles

98. SOLID: five rules that keep object-oriented code easy to change.

- **S – Single Responsibility**: a class has only one job, so only one reason to change. *OrderService handles orders. Sending email goes in a separate EmailService.*
- **O – Open/Closed**: open to extension, closed to modification. Add new behaviour with new code instead of editing working code. *To support UPI, add a* `UpiPayment implements Payment` *class instead of another if-else.*
- **L – Liskov Substitution**: a child class must be usable anywhere its parent is, without breaking anything. *If Penguin extends Bird and Bird has fly(), the code breaks for a penguin. Wrong design.*
- **I – Interface Segregation**: keep interfaces small, so no class implements methods it doesn't need. *Split Worker (work, eat) into Workable and Eatable. A Robot implements only Workable.*
- **D – Dependency Inversion**: depend on interfaces, not concrete classes. *OrderService depends on a Notifier interface, so Email, SMS or a test fake can be plugged in.*

99. Three short rules:

- DRY (Don't Repeat Yourself): write logic once and reuse it. If the same code is in 5 places, a bug must be fixed in 5 places.
- KISS (Keep It Simple, Stupid): choose the simplest solution that works. Simple code is easier to read, test and debug.
- YAGNI (You Aren't Gonna Need It): don't build features "for the future". Build what's needed now.

100. Composition over Inheritance: prefer "has-a" over "is-a". Instead of a deep inheritance chain, build a class out of smaller objects. *A Car has an Engine (so you can swap Petrol for Electric), rather than ElectricCar extends Car extends Vehicle.* Inheritance is rigid; composition is flexible.

101. Dependency Injection (DI) / Inversion of Control (IoC): a class doesn't create its own dependencies with `new`; they're **given** to it from outside.

- **IoC** = the framework (Spring) creates and manages objects instead of your code. **DI** = the way it hands them over (constructor injection is preferred).
- Benefits: loose coupling, easy to swap implementations, easy to test with mocks. *In Spring,* `@Service` *creates a bean and the constructor parameter receives it automatically.*

L. Common Design Patterns

102. Creational (how objects are created):

- Singleton: only **one instance** of a class in the whole app. *DB connection pool, logger, config. Spring beans are singletons by default.*
- Factory: a method decides which class to create, so the caller doesn't use `new` directly. `PaymentFactory.get("UPI")` *returns a UpiPayment.*
- Builder: builds an object with many optional fields step by step, instead of a constructor with 10 parameters. `User.builder().name("Vishal").age(25).build()` *(Lombok @Builder).*

103. Structural (how objects are combined):

- Adapter: converts one interface into another so incompatible code can work together, like a travel plug adapter. *Wrapping a third-party payment SDK to fit your Payment interface.*

- Decorator: adds behaviour to an object without changing its class, by wrapping it. *Coffee → MilkDecorator → SugarDecorator, each adding to the cost.*
- Proxy: a stand-in object that controls access to the real one (lazy loading, caching, security). *Spring's* `@Transactional` *works through a proxy, and Hibernate lazy loading uses proxy objects.*

104. Behavioural (how objects communicate):

- Strategy: pick an algorithm at runtime from interchangeable classes. *Pricing: RegularPricing, SalePricing, PrimePricing, all implementing PricingStrategy. Removes if-else chains.*
- Observer: when one object changes, every subscriber is notified automatically. *YouTube subscribers notified of a new video. Same idea as Pub-Sub / SNS (point 26), but inside one app.*

Part 4 – Interview Prep

M. How to Answer in an Interview (keep this on its own page)

105. Step-by-step (~45 min):

- **1. Requirements**: ask questions first. Functional (what should it do?) and Non-functional (how many users? how fast? is consistency or availability more important?).
- **2. Estimation**: roughly calculate requests/sec (QPS) and storage.
- **3. API design**: main endpoints (`POST /shorten` , `GET /{code}`).
- **4. Data model**: main tables, SQL or NoSQL, and why.
- **5. High-level diagram**:
  `Client → DNS → CDN → LB → API Gateway → Services → Cache → DB / Queue`
- **6. Deep dive**: go deep on one or two parts (scaling the DB, caching, queues).
- **7. Trade-offs**: SPOFs, bottlenecks, what happens when something fails.

106. Quick estimation numbers: 1 day ≈ 100,000 sec, so **1M requests/day ≈ 12 per second**. Peak traffic ≈ 2–3× the average. Most apps are **read-heavy** (100 reads per write), so add a cache and read replicas.

N. Classic Problems – key points to mention

Problem	Key concepts
URL Shortener	Unique ID → Base62 short code, cache popular URLs, redirect (301 / 302)
Rate Limiter	Token bucket / sliding window, counters in Redis, HTTP 429
Chat App (WhatsApp)	WebSockets, queue for offline users, Cassandra for messages, read receipts
News Feed (Instagram)	Precompute feeds for normal users; fetch celebrity posts at read time; cache; cursor pagination
Notification System	SNS → SQS fan-out, retries, DLQ, user preferences
YouTube / Netflix	Upload to S3 → queue → workers convert to many resolutions → serve via CDN
Uber / Zomato	Geohash for nearby drivers, WebSocket location updates
Autocomplete	Trie + top suggestions cached per prefix

Payment System	Idempotency key, ACID DB, Saga for refunds

O. Quick Comparison – what each one solves

Concept	Problem it solves
SOLID / DRY / KISS	Code that is hard to change and maintain
Dependency Injection	Tight coupling, hard-to-test classes
Saga	Transactions across microservices
Outbox	DB write and event publish getting out of sync
CQRS (pt 72)	Read and write loads needing different models
Circuit Breaker (pt 82)	One failing service bringing down others
Strangler Fig	Safe monolith → microservices migration
API Gateway / BFF	Clients calling many services directly